

Cross-Layer Transcoders as Hypothesis Generators for Causal Circuit Analysis in Mathematical Reasoning

Transcoder Explore Project

August 16, 2026

Abstract

Cross-layer transcoders (CLTs) provide a sparse feature basis for transformer MLP computation and can be used to build feature-level attribution graphs. This paper draft organizes an end-to-end case study on Gemma-3-4B-IT: a trained CLT is used to discover candidate features for mathematical discourse, and one feature, L22:F31850, is then tested through static and generation-time interventions. The main empirical lesson is methodological. CLT attribution graphs are useful hypothesis generators, but Neumann-style graph attribution scores do not directly equal causal intervention effects. After isolating nonlinearities from JumpReLU/TopK masks and LayerNorm, L22:F31850 remains a robust causal feature, but its role is better described as a period-position logical-connective continuation axis than as a narrow “However” feature. During AIME-style chain-of-thought generation, scaling this feature changes next-token distributions by suppressing newline/stop behavior and promoting continuation tokens such as `We`, `Since`, and `So`. Aggregate AIME accuracy effects are modest and floor-limited, while per-problem effects are heterogeneous. Quantitatively, at sentence boundaries the edit converts the model’s default stop/newline (the top-1 next token at 66% of periods) into a logical-connective continuation (57%); because this push to keep reasoning is uniform across problems, it helps those that otherwise conclude a step too early (Problem 12: 17% $\rightarrow$ 41% at $\text{sf} = 3$) and hurts those that reward committing to an answer (Problem 23, where suppressing the feature helps instead). This draft is a scaffold for a full research paper; numerical claims are drawn from the local investigation notes and should be audited against raw artifacts before submission.

1 Introduction

Language models learn solution strategies from data and encode them in largely opaque, high-dimensional computations, making it difficult to determine how their capabilities are implemented. Mechanistic interpretability addresses this problem by explaining model outputs in terms of internal computation. Existing approaches operate at several levels: residual-stream and attention analyses trace information through architectural components [8]; ablation, activation patching, and targeted editing test whether components are causally necessary [6, 18, 21]; and dictionary-learning methods decompose dense activations into sparse features that can be described in human terms [5, 23]. Transcoders are particularly useful for circuit analysis because they reconstruct a component’s output from its input, turning an MLP computation into a set of interpretable feature units [7].

Recent work combines *cross-layer transcoders* (CLTs) with *attribution graphs* to describe model computation as prompt-specific circuits of sparse features. Ameisen et al. [1] train CLTs to approximate transformer MLP computations and use them to construct an interpretable local replacement model. Attribution graphs derived from this model represent active features as nodes and direct linear attributions between them as edges. Their methodology also includes graph

pruning and visualization for interpreting these circuits, together with perturbation experiments that test graph-derived interactions and evaluate the replacement model’s mechanistic faithfulness. Lindsey et al. [15] apply this framework to Claude 3.5 Haiku across behaviors including multi-step reasoning, planning, multilingual processing, and hallucination.

[TODO]

Our main contributions are as follows:

- We develop a parameter-efficient latent-mixing CLT for Gemma-3-4B-IT [11], providing a sparse feature basis for circuit analysis.
- We construct attribution graphs for factual and mathematical reasoning and assess their use for candidate discovery. The graphs surface meaningful features, while Neumann scores do not directly recover measured single-feature effects.
- We characterize L22:F31850 as a sentence-boundary continuation axis that strongly changes reasoning layout, while its effect on AIME accuracy remains modest in aggregate and heterogeneous across problems.

2 Preliminaries: Feature-Level Circuit Discovery

2.1 Motivation: From Residual Streams to Cross-Layer Features

Residual streams analysis confounded by superposition Before a circuit can be drawn, we must decide what its nodes are. The transformer’s own architecture is the obvious place to look. In the standard residual-stream account [8], every attention and MLP sublayer reads from a shared residual state and writes an additive update that all later sublayers can read. This is a precise map of *where* information can move, but it is silent on *what* is computed in interpretable terms. And for that question the architectural units fail at both grains. Attention heads and MLP blocks are too coarse to be semantic units; individual neurons are too entangled, since features live in superposition rather than one-per-neuron [5, 23]. A single residual-stream direction mixes several concepts, and the nonlinear map an MLP computes does not decompose into readable feature-to-feature edges. The units we want are not in the architecture; they have to be learned.

Transcoders supply computational units. Learning the units is exactly what Anthropic’s transcoder method does: it keeps the transformer’s residual stream, attention, and normalization, but re-expresses each MLP through a learned sparse feature model [1]. Which sparse model to learn is not a free choice. A sparse autoencoder reconstructs an activation vector, so it answers a representational question: what is *present* in a residual state. A transcoder instead reconstructs the *output of a model component* from that component’s input [7], so it answers a computational question: how to decompose the computation in the MLP into interpretable *units*. The computational question is the one circuit analysis asks. Concretely, a transcoder rewrites the MLP’s nonlinear input–output map as sparse feature activations followed by decoder writes; once attention patterns, normalization denominators, and feature masks are frozen for a prompt, the replacement is locally linear and direct feature-to-feature effects are well defined [1]. The resulting division of labor runs through the whole paper: the residual stream determines where an intervention can propagate, and the transcoder features supply the units that can be named, ranked, and causally edited.

Cross-layer transcoders removes per-layer fragmentation. One obstacle remains. A plain transcoder gives each layer its own private dictionary, so a concept the model carries through many layers reappears as a different, formally unrelated feature at every depth. A *cross-layer* transcoder removes this fragmentation: a CLT feature reads from the residual stream at its source layer and may write into the reconstructed MLP outputs of that layer and all later ones [1]. A feature thereby becomes a node in a cross-layer computation rather than a layer-local proxy, and an attribution path can run through the one feature whose activation and downstream writes explain the computation, rather than through a chain of per-layer stand-ins. In this paper the feature basis is trained for Gemma-3-4B-IT, a 34-layer, width-2560 decoder-only transformer [11], with $M = 100,800$ features per layer and a structural top- k sparsity cap of 500 active features per token and layer.

2.2 Overview: Candidate Feature Discovery and Intervention with CLTs

The CLT lets us score each feature’s contribution to a prediction. With a CLT in place, we can build an *attribution graph* for a chosen prompt and target logit. Its nodes are the active CLT features, the input tokens, and the output logits, and its edges measure how strongly each node influences another. Following the local-replacement construction of Ameisen et al. [1], the transformer’s nonlinear quantities—attention patterns, normalization denominators, and the sparse feature masks—are frozen at the values they take on the prompt, so that feature-to-feature effects can be read off by backpropagating through the resulting linear graph. We use this same machinery to recover readable circuits [1, 15].

Attribution proposes; intervention verifies. Where this paper is deliberately conservative is in how much such a graph is allowed to claim. An edge weight, or a Neumann path-sum aggregating many edges, is an attribution score *inside a local linearization*; it need not equal the change one would measure by actually intervening on the feature in the full nonlinear model. We therefore treat the attribution graph only as a hypothesis generator and let intervention adjudicate. From this point on, the behavior under study is fixed: at a sentence-ending period in an AIME-style reasoning trace, Gemma-3-4B-IT can stop, emit a newline, or carry the argument forward with a logical connective such as *We*, *However*, *Since*, or *So*, and we look for the feature-level circuits that incentivize the connective continuation. Concretely, the study proceeds in three steps: (i) we train a CLT for Gemma-3-4B-IT and build an attribution graph toward such a connective logit; (ii) we read off the handful of features it ranks highest as candidates for that behavior; and (iii) we put the leading candidate through explicit perturbation experiments—a fixed-prompt linear diagnostic that isolates its direct token-level effect, followed by autoregressive AIME generation that measures its behavioral consequences—keeping a causal claim only where the intervention supports it. This carries down to individual features the discipline the broader circuit-discovery literature applies to candidate subgraphs, which are validated by ablation, patched execution, or masked-model fidelity rather than by saliency alone [3, 6, 21, 29].

Roadmap. The rest of the paper instantiates this pipeline. Section 3 defines the CLT architecture and training objective used to reconstruct Gemma-3-4B-IT’s MLP outputs. Section 4 constructs prompt-specific attribution graphs and explains how Neumann scores are turned into the displayed graphs. Section 5 selects L22:F31850 from recurrent attribution candidates and validates it by direct perturbation, first in a fixed-prompt linear diagnostic and then during autoregressive AIME generation. The case study that survives this pipeline is L22:F31850: attribution surfaces it in *However*-continuation contexts, but intervention shows that its causal role is broader, namely suppressing newline/stop behavior at period positions and promoting continued mathematical prose.

3 Cross-Layer Transcoder Formulation

3.1 Transformer Background

Gemma-3-4B-IT [11] is a decoder-only transformer [20, 28] that predicts the next token at every position of an input sequence $u_1, \dots, u_T$. At each position t it maintains a *residual stream* vector of width d [8], whose state entering layer ℓ we write $\mathbf{h}_{\ell,t} \in \mathbb{R}^d$, with

$$\mathbf{h}_{1,t} = \text{Embed}(u_t).$$

Here Embed maps each vocabulary token to a learned d -dimensional vector. Each of the L layers adds two sublayer outputs to the stream: the attention output

$$\mathbf{o}_{\ell,t}^{\text{attn}} = \text{Attn}_{\ell}(\mathbf{h}_{\ell,1}, \dots, \mathbf{h}_{\ell,t}),$$

which reads the stream at all positions up to t , and the MLP output

$$\mathbf{m}_{\ell,t} = \text{MLP}_{\ell}(\mathbf{x}_{\ell,t}),$$

computed position-wise from

$$\mathbf{x}_{\ell,t} = \mathbf{h}_{\ell,t} + \mathbf{o}_{\ell,t}^{\text{attn}},$$

the stream state after the attention update. After the final layer, an unembedding map turns the final state $\mathbf{h}_{L+1,t}$ into the next-token distribution.

The MLP sublayers are our focus. Substantial evidence indicates that they store much of a transformer’s factual and conceptual knowledge: they behave as key–value memories whose value vectors promote concrete output tokens [12], and localized edits to mid-layer MLPs can rewrite specific factual associations [18]. Yet they are difficult to interpret neuron by neuron. Individual hidden neurons are *polysemantic*, participating in many unrelated behaviors, which is what one expects when a layer represents more features than it has neurons by placing them in superposition [5, 9].

3.2 From Transcoders to Cross-Layer Transcoders

Transcoders. Following prior work [7, 13, 22], a *transcoder* addresses this mismatch with a sparse dictionary model trained to reproduce the output of an MLP sublayer from its input, thereby re-expressing the map $\mathbf{x}_{\ell,t} \mapsto \mathbf{m}_{\ell,t}$ in a more interpretable, approximately *monosemantic* feature basis: each feature corresponds to a single, human-describable concept or behavior, in contrast to the polysemantic raw MLP neurons.

Concretely, a layer- ℓ transcoder encodes the residual input $\mathbf{x}_{\ell,t}$ into a high-dimensional vector of *feature* activations $\mathbf{z}_{\ell,t}$ and decodes those features back into an approximation $\widehat{\mathbf{m}}_{\ell,t}$ of the true MLP output $\mathbf{m}_{\ell,t}$:

$$\mathbf{z}_{\ell,t} = \sigma(\mathbf{W}_{\ell}^{\text{enc}}(\mathbf{x}_{\ell,t} + \mathbf{b}_{\ell}^{\text{pre}}) + \mathbf{b}_{\ell}^{\text{enc}}) \in \mathbb{R}^M, \quad \widehat{\mathbf{m}}_{\ell,t} = \sum_{a=1}^M z_{\ell,t,a} \mathbf{w}_{\ell,a} + \mathbf{b}_{\ell}^{\text{dec}}, \quad (1)$$

where $M \gg d$ is the number of features, $\mathbf{W}_{\ell}^{\text{enc}} \in \mathbb{R}^{M \times d}$ is the encoder matrix, $\mathbf{b}_{\ell}^{\text{pre}} \in \mathbb{R}^d$ is a learned bias that recenters the encoder input, $\mathbf{b}_{\ell}^{\text{enc}}$ and $\mathbf{b}_{\ell}^{\text{dec}}$ are learned encoder and decoder biases, and σ is a sparse activation rule that leaves only a few features active at any position. Each feature a owns a decoder direction $\mathbf{w}_{\ell,a} \in \mathbb{R}^d$, and the parameters are trained to minimize the reconstruction error $\|\widehat{\mathbf{m}}_{\ell,t} - \mathbf{m}_{\ell,t}\|^2$ while the transformer itself stays frozen.

Per-layer dictionaries do not capture cross-layer structure. The plain transcoder of Equation (1) learns an independent feature dictionary at each layer. However, information written to the residual stream can persist and be read or modified by later layers. Separate dictionaries may therefore encode the same underlying feature repeatedly at different depths, producing attribution graphs with duplicate nodes connected by mostly uninformative identity-like edges [14]. Such graphs make it difficult to distinguish the persistence of an existing feature from the creation of a new computation.

Cross-layer transcoders. A *cross-layer transcoder* (CLT) [1] addresses this problem by allowing each feature to model effects across multiple layers. Rather than reconstructing only the MLP output at its source layer, a feature encoded at layer s can decode into the MLP outputs of layer s and all later layers. Its downstream influence can therefore be represented by one feature node instead of being rediscovered separately at each layer, yielding a more compact attribution graph.

Formally, a direct CLT extends Equation (1) by allowing a source-layer feature to decode into all downstream MLP outputs:

$$\widehat{\mathbf{m}}_{\ell,t} = \sum_{s \leq \ell} \sum_{a=1}^M z_{s,t,a} \mathbf{w}_a^{s \rightarrow \ell} + \mathbf{b}_{\ell}^{\text{dec}}. \quad (2)$$

Here s indexes source layers, ℓ indexes output (decode) layers, a indexes features, and $z_{s,t,a}$ is the feature activation of Equation (1) computed at source layer s . Every pair (s, ℓ) with $s \leq \ell$ owns its own decoder direction $\mathbf{w}_a^{s \rightarrow \ell}$, allowing one feature to make layer-specific contributions across depth without being duplicated at every layer.

3.3 Latent-Mixing CLT

Parameter cost of the direct CLT. Ignoring biases, the L encoder matrices require LMd parameters. The direct decoder assigns an $M \times d$ block to every source–output pair (s, ℓ) with $s \leq \ell$, and therefore requires

$$\sum_{s=1}^L (L - s + 1)Md = \frac{L(L+1)}{2}Md$$

parameters [25]. For Gemma-3-4B-IT [11], with $L = 34$, $d = 2560$, and $M = 100,000$, the encoder and direct decoder contain 8.704 billion and 152.32 billion parameters, respectively, for 161.024 billion parameters in total, excluding biases, making the direct parameterization prohibitively expensive to train.

Latent mixing. To make cross-layer decoding tractable, our design introduces a shared M -dimensional latent representation in which source-layer features are mixed before being mapped to each output layer by a single decoder (Figure 1). Concretely, the feature activation vectors $\mathbf{z}_{s,t} \in \mathbb{R}^M$ from all source layers use the same latent coordinates. For each output layer ℓ , the vectors with $s \leq \ell$ are mixed featurewise to form an output-specific latent representation $\tilde{\mathbf{z}}_{\ell,t} \in \mathbb{R}^M$:

$$\tilde{\mathbf{z}}_{\ell,t} = \sum_{s \leq \ell} \gamma^{s \rightarrow \ell} \odot \mathbf{z}_{s,t}, \quad \gamma^{s \rightarrow \ell} \in \mathbb{R}^M, \quad (3)$$

where $\odot$ denotes elementwise multiplication. Finally, an output-layer base decoder maps this latent representation to the reconstructed MLP output:

$$\widehat{\mathbf{m}}_{\ell,t} = \mathbf{W}_{\ell}^{\text{dec} \top} \tilde{\mathbf{z}}_{\ell,t} + \mathbf{b}_{\ell}^{\text{dec}}, \quad \mathbf{W}_{\ell}^{\text{dec}} \in \mathbb{R}^{M \times d}. \quad (4)$$

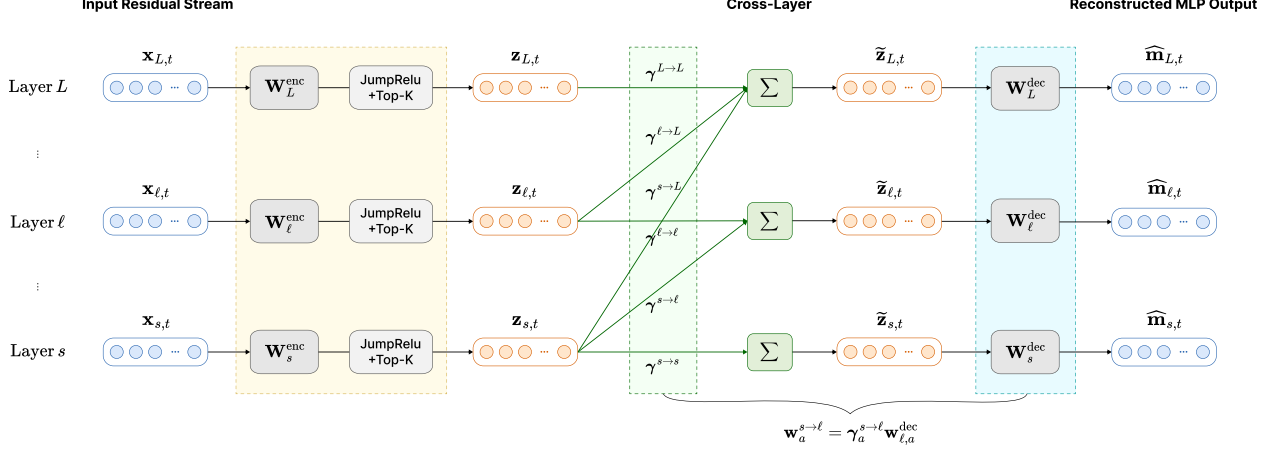


Figure 1: Latent-mixing CLT architecture. Each layer’s residual $\mathbf{x}_{s,t}$ is projected by a layer-specific encoder $\mathbf{W}_s^{\text{enc}}$ into a shared M -dimensional latent coordinate system. Featurewise cross-layer coefficients $\gamma^{s \rightarrow \ell}$ then mix the source-layer activations into each output layer $\ell \geq s$, where an output-layer base decoder $\mathbf{W}_\ell^{\text{dec}}$ reconstructs the MLP output $\widehat{\mathbf{m}}_{\ell,t}$ (equations (3)–(4)).

Latent mixing replaces each independent source–output decoder vector with a learned scalar and an output-layer-specific base direction:

$$\mathbf{w}_a^{s \rightarrow \ell} = \gamma_a^{s \rightarrow \ell} \mathbf{w}_{\ell,a}^{\text{dec}}, \quad (5)$$

Here $\mathbf{w}_{\ell,a}^{\text{dec}}$ is row a of the output-layer base decoder $\mathbf{W}_\ell^{\text{dec}}$ and is shared across source layers, while $\gamma_a^{s \rightarrow \ell}$ controls the contribution of feature a from source layer s to output layer ℓ . Because these coefficients are feature-specific, different features can retain different cross-layer contribution profiles.

	Direct CLT	Latent-mixing CLT
Decoder form	$\mathbf{w}_a^{s \rightarrow \ell}$	$\gamma_a^{s \rightarrow \ell} \mathbf{w}_{\ell,a}^{\text{dec}}$
Interpretation	Independent direction for each (s, ℓ, a)	Shared output-layer direction with source-specific strength
Decoder parameters	$\frac{1}{2} L(L+1)Md = 152.32\text{B}$	$LMd + \frac{1}{2} L(L+1)M \approx 8.764\text{B}$ (17.4× smaller)

Table 1: Comparison of decoder parameterizations. Counts use $L = 34$, $d = 2560$, and $M = 100,000$ and exclude biases.

Table 1 summarizes the trade-off: latent mixing preserves cross-layer contributions while reducing the decoder parameter count by a factor of 17.4, but constrains contributions from different source layers to a given output feature to share the same decoder direction and differ only in scalar strength.

3.4 Training Pipeline

Training objective. Training the CLT is a pure regression problem: at every token position and every layer, reproduce the frozen transformer’s MLP output from its MLP input. Each training step samples B token instances and minimizes the per-coordinate mean-squared reconstruction error

over all L layers,

$$\mathcal{L} = \frac{1}{B L d} \sum_{t=1}^B \sum_{\ell=1}^L \|\widehat{\mathbf{m}}_{\ell,t} - \mathbf{m}_{\ell,t}\|_2^2. \quad (6)$$

We exclude chat-template tokens and padding from the loss because their repetition makes them trivially reconstructable and heavily over-represented, causing the MSE to reward template scaffolding rather than content computation. These tokens still pass through the model normally and are masked only when computing the loss.

Training data. Because our interventions target chat-formatted mathematical reasoning rather than raw pretraining text [1, 7], we train the CLT on chat-formatted QA data. Prompts are drawn from StackExchange questions parsed from the uncopyrighted Pile [10], and the frozen Gemma-3-4B-IT writes each answer in its own chat format [11]. The resulting corpus contains 6,398,447 question-answer pairs and roughly 5.5 billion content tokens. A StackExchange-derived example pair, rendered in the same format the frozen Gemma-3-4B-IT consumes:

```
<bos><start_of_turn>user
Using the M-test, how can I justify differentiating the series  $\sum_{n=1}^{\infty} \lambda^n/n!$  term by
term?<end_of_turn>
<start_of_turn>model
Fix a bounded interval  $0 \leq \lambda \leq R$ . Then each derivative term is bounded by
 $nR^{n-1}/n! = R^{n-1}/(n-1)!$ , and the resulting majorant series converges ...<end_of_turn>
```

The chat-template tokens (<start_of_turn>, <end_of_turn>, and the role labels user and model) are excluded from the reconstruction loss.

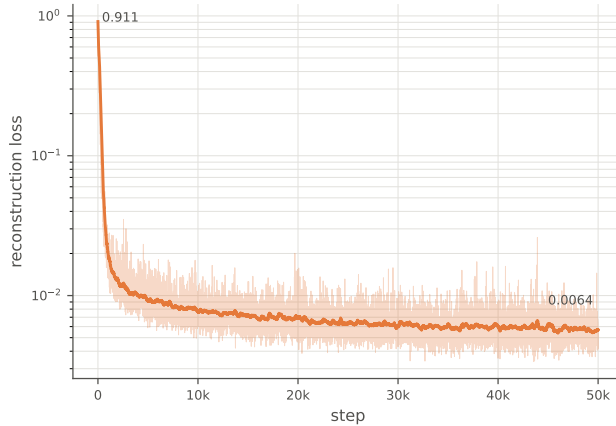


Figure 2: Training trajectory of the analyzed CLT.

Implementation. Training pairs are extracted with forward hooks on the frozen Gemma-3-4B-IT’s MLPs: each unmasked token contributes the residual input $\mathbf{x}_{\ell,t}$ and MLP output $\mathbf{m}_{\ell,t}$ across all layers [24, 26]. Feature activations are sparsified by a JumpReLU with threshold $\theta = 0.025$, followed by a global top- k gate that keeps the $k = 500$ largest features per token and layer across the tensor-parallel shards. We train the analyzed 100,800-feature latent-mixing CLT with AdamW (constant learning rate 10^{-4} , weight decay 10^{-2} , PyTorch-default betas, seed 42) at $B = 64$ tokens per step, on 8 NVIDIA A100 80 GB GPUs with one rank generating these activations and seven training (1.96 steps per second). The reconstruction loss falls from 0.911 at initialization to 0.0064 by step 50,000 (Figure 2); with the loss converged, we stop training there, after about 7.1 hours.

3.5 Why the Training Is Effective

A CLT is only useful as a hypothesis generator if it is a *faithful* and *interpretable* surrogate for the MLP computation it replaces. We assess this along three intrinsic axes: reconstruction fidelity, sparsity, and feature interpretability.

Reconstruction generalizes across evaluation distributions. We evaluate the trained CLT on held-out StackExchange, AIME 2024–2025, NaturalReasoning, Medical-o1, and WikiQA. Across these five distributions, reconstruction MSE ranges from 0.0059 to 0.0083, and pooled relative L_2 error from 0.29 to 0.36 (Table 2). The consistency of these errors indicates that reconstruction transfers beyond StackExchange-style training data. Reconstruction quality varies with depth: the relative L_2 error $\|h_\ell - \hat{h}_\ell\|_2 / \|h_\ell\|_2$ is inverted-U-shaped across layers, rising from 0.15 at layer 0 to 0.67 at layer 13 before falling to 0.28 at layer 33 (Figure 3). This peak coincides with a trough in the activation norm itself, so the middle layers are the thinnest rather than the worst reconstructed in absolute terms. The same profile appears across datasets, indicating a consistent weakness of the surrogate at middle layers rather than dataset-specific degradation.

Table 2: CLT reconstruction on evaluation distributions.

Evaluation set	Pairs	MSE	rel. L_2
StackExchange (held-out)	200	0.0059	0.30
AIME 2024+2025	60	0.0060	0.29
NaturalReasoning	50	0.0063	0.31
Medical-o1	50	0.0082	0.36
WikiQA	50	0.0083	0.35

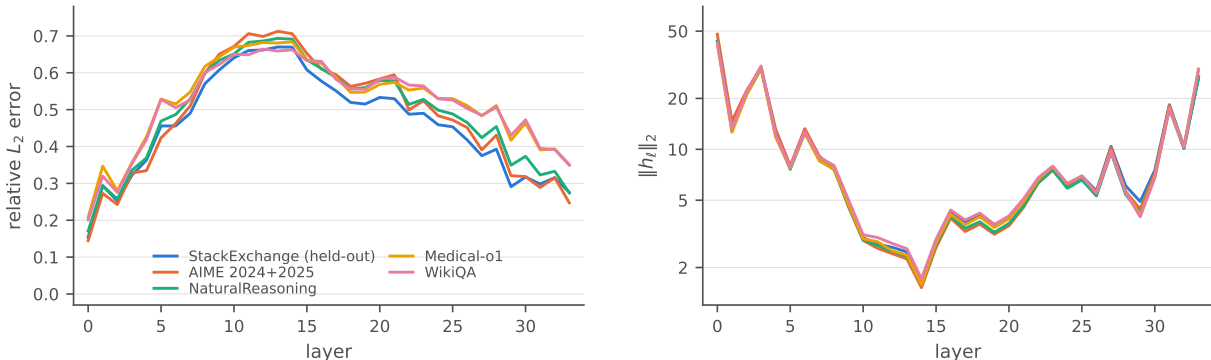


Figure 3: Left: relative L_2 reconstruction error $\|h_\ell - \hat{h}_\ell\|_2 / \|h_\ell\|_2$ by layer. Right: the activation norm $\|h_\ell\|_2$ (log scale), whose trough coincides with the error peak.

Activations are sparse. On the held-out set, a token activates only 58.6 of the 100,800 features per layer on average, a density of 0.06% (Figure 4). Most layers are even sparser: the median layer activates 10.4 features, and 24 of the 34 layers stay below 20 active features per token. The mean is pulled upward by the deepest layers, where the count rises toward the top- k cap of 500. Thus each

MLP output is reconstructed from a small, enumerable set of active features rather than a dense mixture of the full dictionary, making downstream attribution graphs tractable.

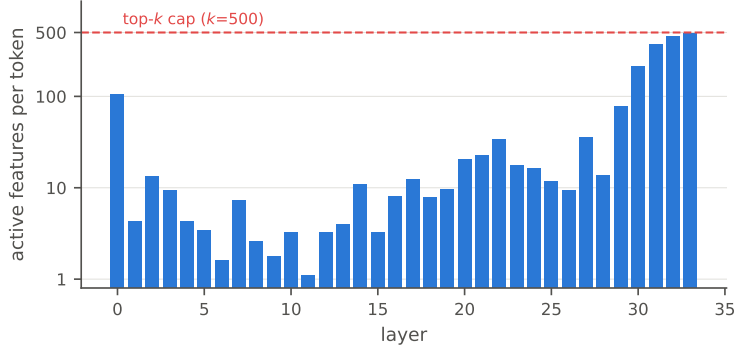


Figure 4: Active features per token (the mean hard ℓ_0) by layer, on the held-out set.

Taken together, our reconstruction error and activation sparsity are comparable to those reported for Anthropic’s production-scale CLTs [1].

Features are interpretable. Faithful, sparse reconstruction supports circuit analysis only if the individual feature directions are themselves interpretable. We run the CLT over a random sample of 20,000 OpenWebMath [19] passages and, for every one of the $34 \times 100,800 \approx 3.4\text{M}$ features, record its firing rate together with the ten passages that most strongly activate it and the token position of each peak. Selection then follows from statistics, not manual taste: we discard the first-token artifact, retain those with a non-trivial peak (maximum activation at least 4 in our normalized activation units), and label each retained feature by automated interpretability [4], prompting a language model to describe what its maximally-activating examples share.

Table 3 reports six retained features chosen to span the dictionary’s depth (layers 0 through 31) and the kinds of structure it learns: early-layer lexical concepts, mid-layer formatting and reasoning features, and deep-layer *register* features. Across the three top passages shown for each feature, the automated labels remain coherent.

Table 3: Automated-interpretability evidence that CLT features are monosemantic. Features are denoted $L\ell:F_a$, where ℓ is the layer and a the feature index. Token backgrounds are shaded in proportion to activation strength, with darker colors indicating larger activations.

Feature	Label	Top-activating OpenWebMath excerpts (token tint $\propto$ activation)
L0:F30117	Enterprise-software names	### Moving from MediaWiki to SharePoint O365 - part 4 · This is an example of SharePoint list lookup column in PowerApps. Power intranet local site, and sites in the intranet are configured (this is necessary in order ...
L0:F28233	Troubleshooting & diagnosis	... errors out ...anyone willing to PM to troubleshoot ? • rooted, took me a the Design and Analysis of Experiments for process troubleshooting and discovering ways to improve process output. prompt. I do this because a) troubleshooting database issues is far outside of my skillset ...
L20:F14399	Section headings	... ## Stochastic Oscillator 101 The fundamental premise ## Defining Definite Integrals What is a definite ## Countable and Uncountable Sets Definition. ...
L22:F31850	Mathematical logic/proof	... show that m forms a basis . However , I am unsure as to whether set $f(U)$ is open. I saw related questions where other users a mono- and an epimorphism. Note that the usual examples fo
L30:F38145	Academic citation	... theorem in the theory of binary relations }, journal = { Compositio Mathematica }, illi }, journal = { Celestial mechanics }, year = { 1985 }, }, journal = { J. Complex. }, year = { 2004 }, ...
L31:F31060	Problem-solving write-ups	... Relevant equations 3. The attempt at a solution I don't know where }) 3. The attempt at a solution Going back to basics with this 7:53 There is a tricky substi

4 Attribution as Candidate Discovery

To make the goal of this section concrete, consider the prompt

“... it is easy to show that m forms a basis.”

which Gemma-3-4B-IT continues with the token **However**. In this case the model could have stopped, started a new line, or kept explaining in some other way; instead it produced a discourse connective that carries the argument across the sentence boundary. To find the feature-level circuit behind that choice, we build an *attribution graph* toward the **However** logit.

High-level workflow. Given a prompt and target logit, we first construct a prompt-specific graph of direct influences among input tokens, CLT features, and the output. A truncated Neumann series then aggregates paths through intermediate features into total influence scores. These scores determine which feature nodes are displayed, while the direct-effect edges among the displayed nodes form the compact attribution graph.

4.1 Direct Effect Computation

The first step toward the attribution graph is computing its edges: the *direct effect* of one node on another, i.e. how strongly a source node (a feature or an input token) pushes on a target node’s readout (a downstream feature’s pre-activation, or an output logit) in a single step, before any routing through intermediate features.

We compute these direct effects in a locally linearized *replacement model*: every MLP sublayer is replaced by the trained CLT, while the remaining nonlinearities (attention probabilities, RMSNorm scale factors, and the JumpReLU/TopK active-feature masks) are frozen at their forward-pass values and gradients are not allowed to propagate through them. We write $g_{i \rightarrow j}$ for the direct effect from source node i to target node j , computed as the corresponding gradient in this frozen model. The gradient is also blocked from passing through any other CLT feature’s encode–decode chain, so influence routed through a third feature is excluded. This procedure yields a direct-effect value for each admissible source–target pair.

Concretely, let $\mathbf{J}_{i \rightarrow j}$ denote the transformer Jacobian that propagates a residual-stream perturbation from the source location to the target location. For a feature-to-feature edge, the implementation composes the activation-weighted source decoder write, this Jacobian, and the target encoder read:

$$g_{i \rightarrow j} = \mathbf{w}_j^{\text{enc}} \mathbf{J}_{i \rightarrow j} (z_i \mathbf{w}_i^{\text{dec}}). \quad (7)$$

Read from right to left, the decoder writes source feature i into the residual stream, the Jacobian carries that write forward, and the encoder reads out target feature j (Figure 5). Token and logit endpoints replace the decoder or encoder with the corresponding embedding or unembedding direction. Appendix B gives the cross-layer sums and full implementation details.

Original Gemma-3-4B-IT transformer

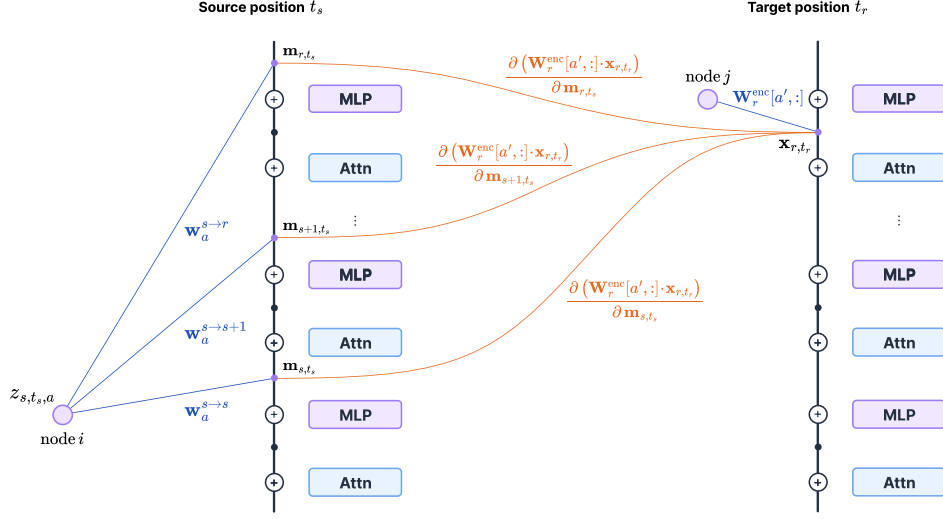


Figure 5: Direct-effect computation. A source feature’s decoder direction is propagated through the transformer Jacobian and projected onto the target feature’s encoder direction. Repeating this computation over source nodes fills one target row of the attribution matrix.

Collecting these pairwise direct effects gives a prompt-specific adjacency matrix $\mathbf{G}$. We enumerate the graph nodes for the prompt and arrange their pairwise direct effects in a target-by-source matrix, with $(\mathbf{G})_{j,i} = g_{i \rightarrow j}$. Target rows are ordered as [feature; logit] and source columns as [token; feature], so the graph adjacency has four main blocks: feature targets reading from token and feature sources, and logit targets reading from token and feature sources. With each block written target-by-source, $(\mathbf{G}_{X \leftarrow Y})_{x,y} = g_{y \rightarrow x}$, the adjacency matrix is

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_{F \leftarrow T} & \mathbf{G}_{F \leftarrow F} \\ \mathbf{G}_{L \leftarrow T} & \mathbf{G}_{L \leftarrow F} \end{pmatrix} \begin{array}{l} \leftarrow \text{feature rows} \\ \leftarrow \text{logit rows.} \end{array} \quad (8)$$

4.2 Neumann Multi-Hop Propagation

The direct-effect matrix $\mathbf{G}$ records pairwise, single-step influence. We use Neumann-series propagation to aggregate these edges over paths of different lengths. To compose multi-hop paths, we distinguish edges by their target type: edges into logits terminate propagation, whereas edges into features can be followed by additional edges. We define

$$A = (\mathbf{G}_{L \leftarrow T} \quad \mathbf{G}_{L \leftarrow F}) \in \mathbb{R}^{n_L \times n_{\text{tf}}}, \quad B = \begin{pmatrix} \mathbf{0}_{n_T \times n_T} & \mathbf{0}_{n_T \times n_F} \\ \mathbf{G}_{F \leftarrow T} & \mathbf{G}_{F \leftarrow F} \end{pmatrix} \in \mathbb{R}^{n_{\text{tf}} \times n_{\text{tf}}}. \quad (9)$$

where $n_F = N_{\text{feat}}$, $n_T = N_{\text{token}}$, $n_L = N_{\text{logit}}$, and $n_{\text{tf}} = n_T + n_F$. The matrix A contains direct effects from token and feature sources to logit targets, while B contains direct effects from the same sources to feature targets; its zero token rows encode that input tokens can begin a path but cannot be reached from another node.

The path contributions are then generated by repeated multiplication: let $T_0 = A$ and $T_i = T_{i-1}B = AB^i$ for $i \geq 1$. Each multiplication by B extends a path by one more intermediate feature, so the entry $(T_i)_{\ell,s}$ collects every path that runs from source s to logit ℓ through exactly i intermediate features. In particular T_0 is the *direct* contribution of a source to the logit, and T_i with $i \geq 1$ is its *indirect* contribution through i hops.

These hop terms shrink quickly in practice. Figure 6 plots the total magnitude $\|T_i\|_1$ of each hop, i.e. the sum of the absolute values of all entries of T_i . After the first two hops the terms fall off geometrically, each about a quarter of the previous one, from $\|T_0\|_1 \approx 6.4 \times 10^3$ to $\|T_{10}\|_1 \approx 0.14$, so the multi-hop sum is dominated by the first few hops.

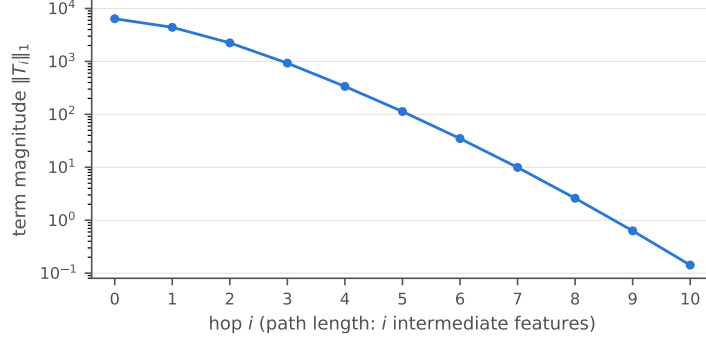


Figure 6: Geometric decay of the Neumann-series terms. Each point is the total magnitude $\|T_i\|_1$ of the hop- i contribution on the full attribution graph of the `However` prompt.

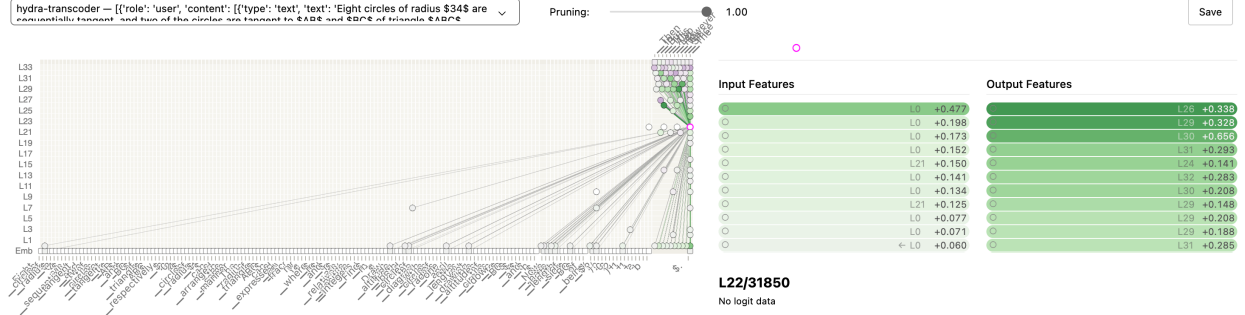
Because of this decay, truncating at a finite order n already captures essentially all of the multi-hop influence. Summing the contributions up to that order gives the *total* influence score

$$S_n = \sum_{i=0}^n T_i = A + AB + AB^2 + \cdots + AB^n. \quad (10)$$

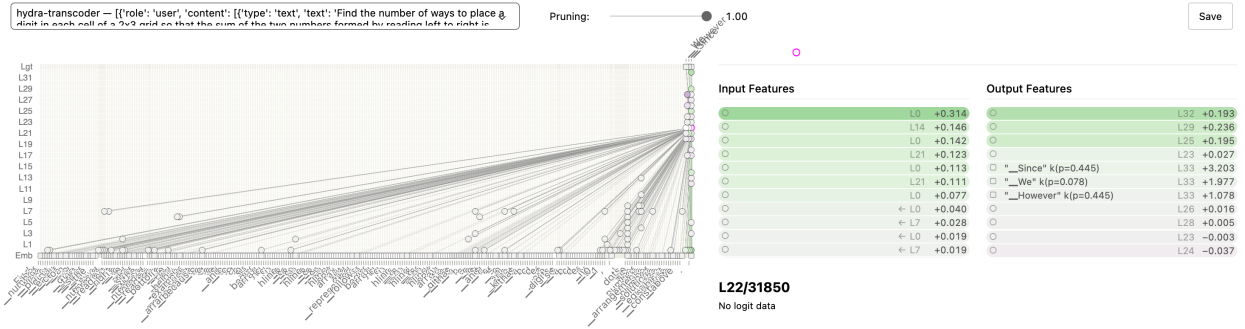
All attribution scores in this paper are computed with $n = 10$: the last term kept, T_{10} , is already more than four orders of magnitude smaller than T_0 , so the terms dropped beyond it are negligible.

4.3 From Neumann Scores to Attribution Graphs

To make the scores easier to inspect, we visualize them as a compact attribution graph. For a fixed prompt and target continuation logit, equation (10) gives a total influence score from every token or feature source to that logit. We retain the 100 CLT features with the largest positive Neumann scores toward the target logit and assemble them into a directed graph. The direct-effect matrix $\mathbf{G}$ supplies the edges among these nodes, while the Neumann score determines how strongly each source is attributed to the target after accounting for paths through intermediate features.



(a) AIME 2024, Problem 5 (“eight circles” geometry).



(b) AIME 2024, Problem 12 (“2x3 grid” combinatorics).

Figure 7: Attribution graphs for two AIME discovery sentences. The horizontal axis follows token position: input tokens appear along the bottom, the target continuation logit is placed at the top, and CLT feature nodes sit between them at the layer and token position where they activate. Gray links show positive direct effects among the retained nodes. The side panel summarizes the selected feature’s local neighborhood: input-feature bars show which upstream features most strongly feed into it, and output-feature bars show which downstream features or logits it most strongly influences.

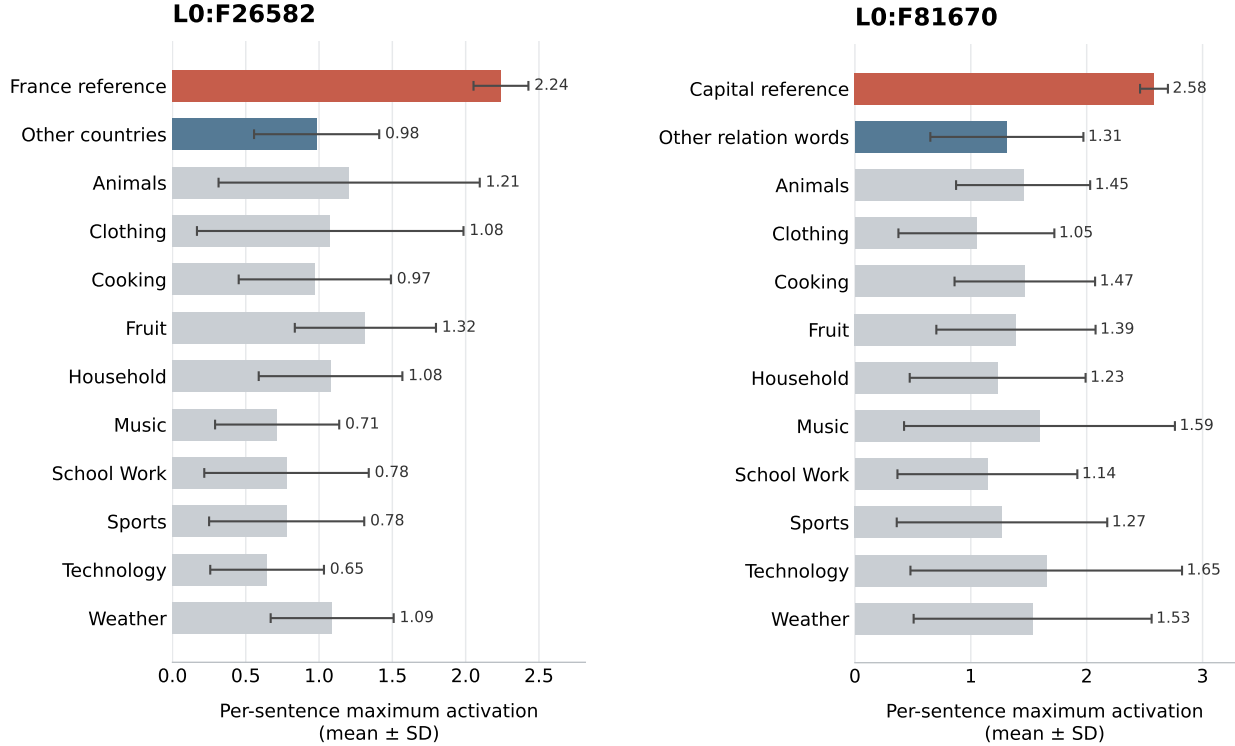
4.4 A Worked Example: From an Attribution Graph to a Partial Causal Trace

To make the graph-to-explanation workflow in Section 4.3 concrete, we study the factual completion **The capital of France is → Paris**. Graph attribution nominates influential features and links, controlled activation tests their provisional semantic roles, and intervention tests a candidate path through them. Our goal is to recover one supported circuit fragment, not an exhaustive or unique account of the computation.

[TODO: Add the full causal-tracing attribution graph for **The capital of France is → Paris before presenting the semantic screen. The figure should show the target logit, the three screened source positions, and the retained feature paths so readers can see how the candidates arise from the Section 4.3 pipeline.]**

Graph-based candidate selection and activation specificity. From the prompt-specific attribution graph, we retain CLT features at the literal **capital**, literal **France**, and final **is** positions whose aggregated Neumann attribution exceeds 0.01, yielding 1,034 candidates. Controlled changes in wording, entities, and relations then test whether their activation patterns match a proposed role in the trace **France/capital → answer state → Paris**.

Figure 8 reports representative activation evidence for the two input features retained after graph



(a) L0:F26582: France references versus other-country and unrelated topics.

(b) L0:F81670: capital references versus alternative-relation and unrelated topics.

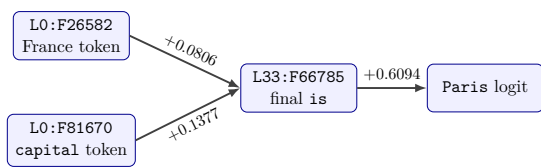
Figure 8: Controlled activation evidence for the two input features retained in the final circuit trace. Each bar is the mean across sentences of the maximum feature activation observed in each sentence within the indicated topic; error bars show one standard deviation across sentences.

attribution, activation screening, and ablation: L0:F26582 at **France** and L0:F81670 at **capital**. Appendix A gives the complete prompt suite and selection procedure.

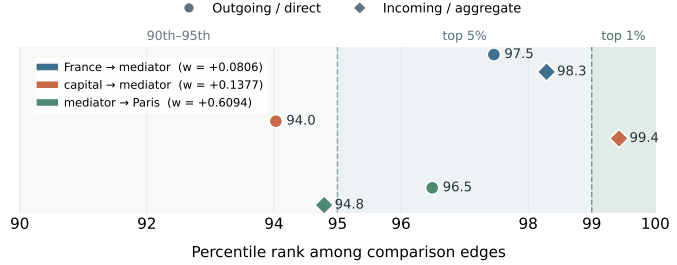
France token. The screen identifies three layer-0 candidates. Among them, targeted ablations identify L0:F26582 as the clearest individual contributor to the **Paris** output, while joint ablation indicates distributed input support. Across 24 France sentences, 24 other-country controls, and 120 unrelated sentences, **France** is its maximally activating token in 23/24 France contexts, and sentence maxima separate from unrelated text with AUC 0.972 (Figure 8(a)). Its weaker responses to unrelated text motivate the narrower description *literal-France feature*, rather than France-only detector.

Capital token. Among six high-attribution candidates tested against seven relation controls, L0:F81670 ranks second and third by direct and multi-hop **Paris** attribution among 155 features, and its ablation lowers $P(\text{Paris})$ by 0.0446. Across 24 capital-reference, 28 alternative-relation, and 120 unrelated sentences, it peaks at **capital** in every capital context (Figure 8(b)), although some unrelated words activate it more strongly. L0:F20862 is more activation-selective but does not support the **Paris** logit, so we retain L0:F81670 as the literal-capital input feature.

Partial shared mediator. We next search for a shared downstream feature that receives positive graph edges from both retained inputs, responds to their ablation, and positively supports **Paris**. The strongest candidate is L33:F66785 at the final **is** position (Figure 9).



(a) The proposed trace fragment from the two literal-input features through a shared mediator.



(b) Each link’s signed attribution weight and percentile rank within its matched local edge distributions.

Figure 9: Graph-supported candidate trace and local edge-strength context for the **Paris** output.

Figure 9(b) shows that the proposed edges are strong relative to their local backgrounds. The France and capital edges rank at the 97.46th and 94.03rd percentiles among their sources’ positive outgoing edges, and at the 98.29th and 99.43rd percentiles among positive edges entering the mediator.

Intervention supports L33:F66785 as a causally relevant shared downstream feature: ablating either input reduces its activation, while ablating the mediator changes the **Paris** logit by -0.0607 . This evidence supports the candidate partial trace $L0:F26582/L0:F81670 \rightarrow L33:F66785 \rightarrow \text{Paris}$, but not a dominant mediated pathway. Appendix A.4 reports the path-share and targeted-rescue bounds.

5 Selecting and Validating the Target Feature

This section looks for the CLT feature that actually drives the behavior introduced at the start of Section 4: after a sentence-ending period in mathematical reasoning, the model continues with a logical connective instead of stopping or starting a new line.

Feature Discovery. Feature discovery runs Neumann propagation on six sentences drawn from AIME reasoning traces. Each sentence ends where Gemma-3-4B-IT assigns substantial probability to a logical-connective continuation (**However**, **Since**, and **So**). For each sentence we rank features by Neumann score toward its continuation logit and keep the top 100.

We compare the top-100 lists across all six sentences, identifying each feature by its (layer, feature) pair and dropping token position. This yields five recurrent candidate features. Each appears in the top 100 for at least five of the six sentences.

Recurrence serves as an initial consistency criterion, but selection for detailed study additionally requires semantic interpretability. Each candidate has an automated-interpretability label from Section 3.5, computed on training-corpus top activations and independent of the six sentences. Table 4 reports these labels together with representative maximally activating OpenWebMath examples.

Linearized Fixed-Prompt Interventions. Then we test the candidate features with a scale-factor edit under a linearized fixed-prompt protocol, where we directly change one CLT feature activation at one token position and measure the resulting change in the next-token distribution.

For a feature activation $\mathbf{z}_{\ell,t,a}$ at layer ℓ , token position t , and feature index a , the scale-factor

Table 4: Recurrent feature candidates and their automated-interpretability evidence. Recurrence is the number of discovery sentences for which the feature appears in the Neumann top-100 list. Colored **act** badges encode each passage’s peak activation.

Feature	Recur.	Automated label	Representative top-activating OpenWebMath examples
L22:F31850	5/6	Mathematical logic/proof	act=7.41 : Are there cyclic, free modules where the generating element is not a basis? Let R be a ring, and M a nontrivial module ... act=7.38 : Extension Fields—prove that $\mathbb{Q}(\sqrt{2}, \sqrt{3})$ is isomorphic to ... act=7.13 : Divisibility of Tamagawa numbers—let $E/\mathbb{Q}$ be an elliptic curve of conductor N , and let $p \geq 11$ be prime ...
L29:F60066	5/6	Math exercises	act=4.78 : Home > INT3 > Chapter 12 > Lesson 12.1.4 > Problem 12-63—sketch a graph of each of the following functions ... act=3.69 : Integrate with a formula? The problem statement, all variables and given/known data ... act=3.47 : Solving systems of equations: $r + s + t = 5$, $2r + 7s - 3t = 13$...
L32:F53761	6/6	General academic	act=3.36 : Department of Mathematics—seminar calendar for events on Thursday, February 27, 2020 ... act=3.36 : On the links of vertices in simplicial d -complexes embeddable in Euclidean $2d$ -space ... act=3.13 : CBSE Class 10 Maths Sample Question Paper, 2019 ...
L25:F20384	6/6	STEM Q&A	act=7.72 : Vertical Circular Motion—a roller coaster problem discussed in a physics forum ... act=7.50 : Historical calibration of the Hull–White model—a question concerning the one-factor model ... act=7.47 : Compressible Flow: de Laval Nozzle—overview and aims of the test case ...
L7:F20827	6/6	Conversational openings	act=9.06 : “We’re talking about organizing and maps! Join us on Open Call on May 11 ...” act=9.00 : “It’s a pity, but I’m probably going to nuke too.diego.codes ...” act=8.75 : “What’s Going On”—a comment on a lecture slide about memory consistency ...

intervention multiplies the activation by a constant:

$$\mathbf{z}'_{\ell,t,a} = \text{sf} \cdot \mathbf{z}_{\ell,t,a}, \quad \Delta_{\ell,t,a} = (\text{sf} - 1)\mathbf{z}_{\ell,t,a}. \quad (11)$$

The baseline is $\text{sf} = 1$: $\text{sf} > 1$ amplifies the feature, $\text{sf} = 0$ removes its active contribution, and negative values reverse it. As the perturbation is proportional to the feature’s natural activation, scale factors are comparable across prompts.

The diagnostic holds the prompt tokens fixed and compares the next-token distribution before

and after the edit in a linearized CLT replacement forward. It asks whether the response scales linearly with edit size, and whether the Neumann attribution score predicts the measured slope.

Table 5 compares prediction and intervention for the five candidate features, using the next-token **However** logit on the AIME 2024 Problem 5 sentence as the fixed target. For feature f , the *Neumann gradient* g_f^{Neu} is the graph-predicted derivative per unit activation, and the *observed slope* g_f^{obs} is the measured finite-difference derivative:

$$g_f^{\text{Neu}} = \frac{S_{f \rightarrow \text{However}}^{\text{Neu}}}{\mathbf{z}_f^0}, \quad g_f^{\text{obs}} = \frac{\text{logit}_{\text{However}}(\text{sf}) - \text{logit}_{\text{However}}(1)}{(\text{sf} - 1)\mathbf{z}_f^0}, \quad (12)$$

Here, $S_{f \rightarrow \text{However}}^{\text{Neu}}$ is f 's activation-weighted Neumann score, and $\mathbf{z}_f^0$ is its baseline activation at the selected perturbation position. The quantity $\text{logit}_{\text{However}}(\text{sf})$ denotes the next-token **However** logit obtained when feature f is scaled by sf at that position; $\text{logit}_{\text{However}}(1)$ is the unedited baseline. The ratio $\rho_f = g_f^{\text{obs}}/g_f^{\text{Neu}}$ is the fraction of the graph-predicted effect realized by intervention; $\rho_f < 0$ means the measured effect points opposite to the prediction.

Under this protocol the response scales linearly with the edit, but Neumann scores do not predict causal magnitude. None of the observed-to-predicted ratios approaches one, and two Neumann-positive features have negative measured slopes.

This discrepancy is not a nonlinearity artifact: it persists even in the frozen linear regime. The mismatch is structural: the Neumann score credits f with the entire multi-hop path sum toward the logit, while the measured slope perturbs f alone. When negative routes dominate at the perturbed position, the marginal effect can point opposite to the aggregate score. Attribution scores are rankings, not causal magnitudes, and must be checked by intervention.

Combining the semantic and intervention evidence, we select L22:F31850 for the subsequent generation-time study. Its automated-interpretability label and top-activating examples align most directly with algebraic proof and logical continuation in AIME reasoning. It also has the largest observed-to-predicted ratio among the five candidates, indicating that the greatest fraction of its Neumann-predicted effect is realized under intervention. The selection therefore reflects both semantic relevance and causal support.

Table 5: The attribution graph’s predicted effect versus the measured effect for five candidate features. Each feature is perturbed at its own position, taken from its selected node in this sentence’s attribution graph.

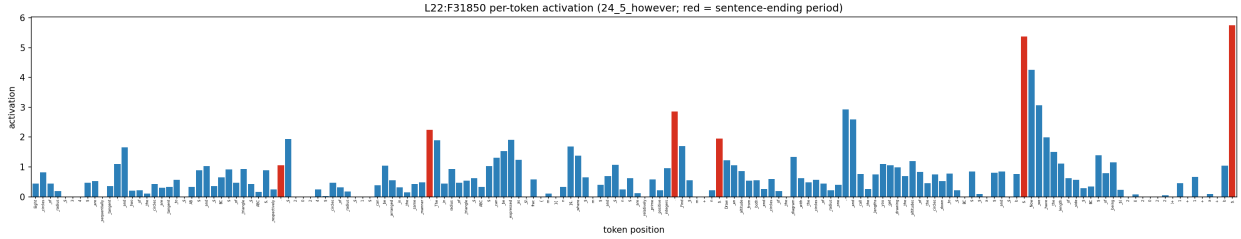
Feature	Neumann grad. g_f^{Neu}	Observed slope g_f^{obs}	Ratio ρ_f
L22:F31850	0.707	0.210	0.30
L29:F60066	1.800	0.192	0.11
L32:F53761	1.636	0.089	0.05
L25:F20384	0.351	-0.064	-0.18
L7:F20827	0.504	-0.027	-0.05

6 Perturbation Experiments

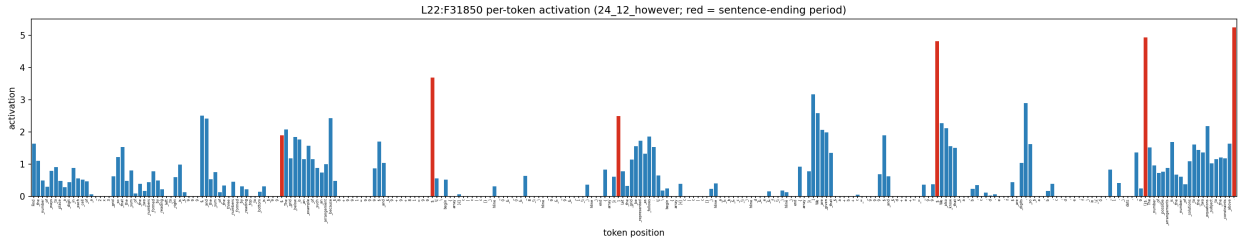
This section investigates what role the selected feature plays when Gemma-3-4B-IT generates AIME solutions autoregressively. We use the same scale-factor edit defined above in equation (11), but apply it during generation.

6.1 Setup

Period-triggered perturbations. L22:F31850 activates particularly strongly at sentence-ending period positions, so we restrict the intervention to those positions. Across the six discovery sentences, L22:F31850’s mean activation on sentence-ending period tokens is $3.1\text{--}8.6\times$ higher than on other tokens, with a pooled ratio of $5.3\times$; Figure 10 illustrates this concentration for two representative sentences. This consistent selectivity provides a natural trigger: during autoregressive generation, every token whose decoded form ends in a period is treated as a candidate intervention position, and the edit is applied when L22:F31850 has positive baseline activation at that position. The resulting protocol focuses the intervention on positions at which the feature is most consistently active.



(a) AIME 2024, Problem 5 (“eight circles” geometry), target **However**.



(b) AIME 2024, Problem 12 (“2x3 grid” combinatorics), target **However**.

Figure 10: Per-token activation of L22:F31850 across two of the six discovery sentences. Each bar is the feature’s activation at one token position; red bars mark tokens whose decoded text ends in a period.

Cache-propagated period interventions. At each period trigger, the runner pauses generation and performs three full-prefix forwards. A clean vanilla forward records the target-feature activation and clean logits, caches the attention softmax patterns and RMSNorm scaling factors, and stores the per-layer MLP outputs used to compute reconstruction errors. A zero-perturbation replacement forward then computes the baseline CLT activations, fixes the JumpReLU/Top- K firing masks, and computes the reconstruction errors that are held fixed during the edit. An edited replacement forward applies the scale-factor edit to L22:F31850 while reusing these cached nonlinear states and captures the resulting K and V vectors for the period position. These perturbed K/V entries are written back into the live autoregressive cache, allowing the intervention to influence subsequent token generation through both attention weights and value aggregation [27].

Implementation. We evaluate on the 30 problems of AIME 2024 and the 30 of AIME 2025 [16], with problems and reference answers taken from public datasets [2, 17]. For each problem and setting, we sample 128 solutions at temperature 1.0 with a 2,048-token generation budget under three settings: the unedited model (**Vanilla**), amplification ($\text{sf}=3$), and reversal ($\text{sf}=-1$).

A solution is scored correct if its final boxed integer equals the reference answer. The edited runs contain 344,124 ($sf=3$) and 319,946 ($sf=-1$) period firings. Comparisons across settings are restricted to the 7,141 problem-sample pairs for which all three settings produced a solution.

6.2 Results

We evaluate the cache-propagated intervention at three levels of granularity: the next-token distributions recorded at each intervened period (Figures 11 and 12), the surface organization of complete sampled solutions (Tables 6 and 7), and final-answer accuracy (Figure 13). Across all three levels the same picture emerges: scaling L22:F31850 acts as a bidirectional control over the stop-versus-continue decision at sentence boundaries, and the resulting accuracy effect is real but problem-dependent.

The perturbation shifts next-token logits. Figure 11 reports the paired change in next-token probability at each intervened period. Because the clean and edited distributions are evaluated on the same prefix, this comparison isolates the immediate effect of the current intervention.

Amplification reduces the combined probability of `\n` and `\n\n` by 29.8 percentage points, from 54.9% to 25.0%. The largest increases occur for `Since` (+14.4 points) and `We` (+10.6), followed by smaller gains for `So`, `Let`, `In`, and `However`. Reversal shifts the distribution in the opposite direction: the combined newline probability rises by 10.3 points, from 75.0% to 85.3%, while `Since` and `We` decrease by 1.8 and 3.0 points. The effect is not a uniform increase in sentence-initial connectives, since `Then`, `Also`, and `Thus` all decrease under amplification. Instead, the perturbation redistributes probability between newline tokens and a restricted set of continuations led by `Since` and `We`.

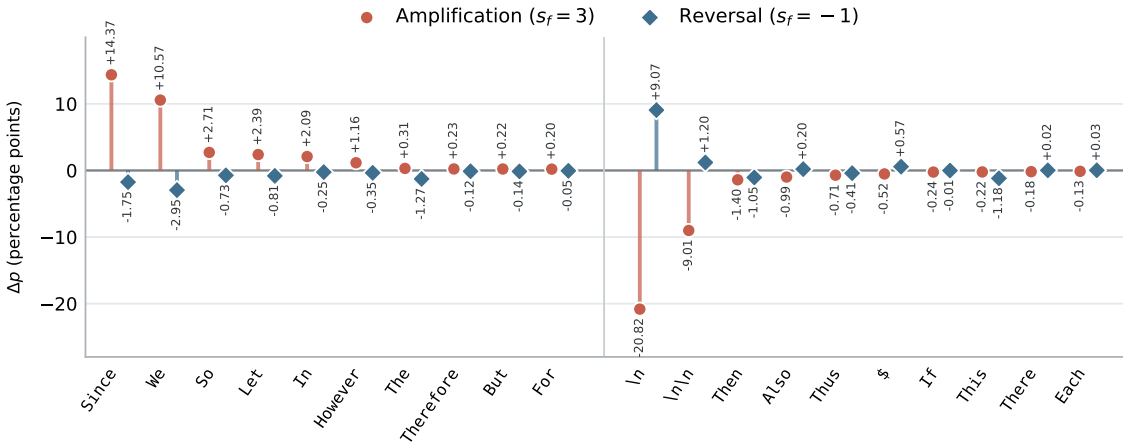


Figure 11: Change in mean next-token probability after intervention at sentence-ending period firings, averaged over all firings of each run and pooled over AIME 2024 and 2025. Tokens are ordered by their change under amplification, and the gap separates the ten most promoted from the ten most suppressed.

Empirical continuation frequencies reflect the probability shifts. Figure 12 reports the empirical frequencies of tokens generated after sentence-ending periods under each setting, complementing the fixed-prefix probability changes in Figure 11. The three settings have comparable

numbers of period continuations, permitting direct comparison of their normalized compositions. Amplification reduces the newline share from 65.8% to 24.4%; separately, `Since` rises from 2.8% to 18.3%, `We` from 4.6% to 17.1%, and the pooled frequency of `So`, `Let`, `In`, and `However` from 4.0% to 14.8%. Reversal raises the newline share to 83.1%, while `Since` and `We` fall independently to 0.1% and 0.5%, and the pooled connective group falls to 0.4%. Thus, the fixed-prefix probability shifts survive sampling and are not tied to a single lexical item: scaling L22:F31850 rebalances line breaks against a restricted family of sentence-initial continuations throughout the autoregressive trajectory.

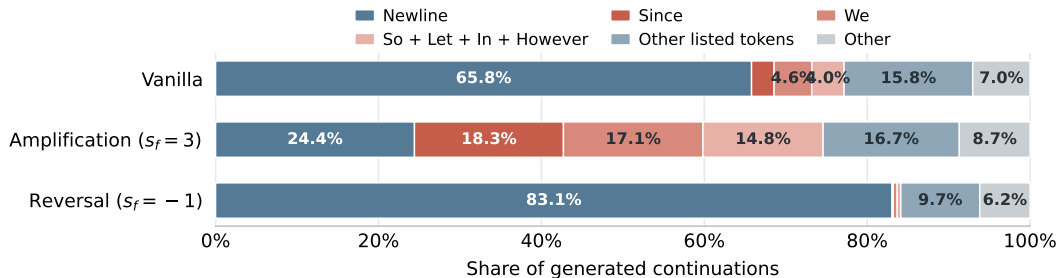


Figure 12: Composition of tokens actually generated after sentence-ending periods in the 7,141 matched AIME 2024 and 2025 solutions. “Other listed tokens” combines `Then`, `The`, `This`, `Thus`, `$`, and `Also`.

The token-level shift compounds into the layout of entire solutions. Compounded over an entire generation, the token-level shift produces a large change in how solutions are laid out on the page. Over the 7,141 matched problem–sample pairs, amplification roughly halves the number of lines per solution ($37.6 \rightarrow 20.0$) and doubles the mean line length ($40.5 \rightarrow 81.8$ tokens), while reversal moves both statistics in the opposite direction (44.2 lines of 33.7 tokens each; Table 6). Table 7 makes this aggregate shift concrete in two matched examples. For AIME 2025 Problem 10, the unedited solution places each step of the case analysis on its own line, whereas amplification ($s_f = 3$) joins each case into a continuous sentence chain that reaches the same roots. For AIME 2025 Problem 21, the corresponding contrast is one fact per line versus connected paragraphs. These examples are the per-solution manifestation of the line-count statistics in Table 6.

Table 6: Line statistics of the generated solutions, averaged over the same 7,141 AIME 2024 and 2025 problem–sample pairs for which all three settings produced a solution.

Setting	Samples	Tokens/solution	Lines/solution	Newlines/1k tokens	Tokens/line
Unedited (vanilla)	7,141	1,526	37.6	27.8	40.5
Amplified ($s_f=3$)	7,141	1,634	20.0	14.5	81.8
Suppressed ($s_f=-1$)	7,141	1,490	44.2	32.8	33.7

(a) AIME 2025, Problem 10: amplification joins the case analysis into sentences.

Unedited (vanilla) — one step per line

We have $x = 34y^2$, and we want x such that $f(x) = y$.
If $x \in [-1, 1]$, then $f(x) = x$, **so** we want $x = y$.
Then $y = 34y^2$, **so** $34y^2 - y = 0$, **which gives**
 $y(34y - 1) = 0$. **Thus** $y = 0$ or $y = \frac{1}{34}$.
If $x \in [1, 3]$, then $f(x) = 2 - x$, **so** we want $2 - x = y$, **so**
 $x = 2 - y$.
Then $2 - y = 34y^2$, **so** $34y^2 + y - 2 = 0$.
 $y = \frac{-1 \pm \sqrt{1 - 4(34)(-2)}}{2(34)} = \frac{-1 \pm \sqrt{1 + 272}}{68} = \frac{-1 \pm \sqrt{273}}{68}$.
Since $y > 0$, we take $y = \frac{-1 + \sqrt{273}}{68}$.

Amplified (sf = 3) — each case one sentence chain

Case 1: $-1 \leq 34y^2 < 1$. **Then** $-1 \leq 34y^2 < 1$, **so**
 $-\frac{1}{34} \leq y^2 < \frac{1}{34}$. **Since** $y^2 \geq 0$, we have $0 \leq y^2 < \frac{1}{34}$, **which**
means $0 \leq y < \sqrt{\frac{1}{34}} = \frac{1}{\sqrt{34}}$. **In** this case, $f(34y^2) = 34y^2$.
We want to solve $34y^2 = y$, **so** $34y^2 - y = 0$, **which gives**
 $y(34y - 1) = 0$. **Thus** $y = 0$ or $y = \frac{1}{34}$. **Since**
 $0 \leq y < \frac{1}{\sqrt{34}} \approx \frac{1}{5.83} \approx 0.17$, both $y = 0$ and $y = \frac{1}{34}$ are valid.
Case 2: $1 \leq 34y^2 < 3$. **Then** $1 \leq 34y^2 < 3$, **so** $\frac{1}{34} \leq y^2 < \frac{3}{34}$,
which means $\frac{1}{\sqrt{34}} \leq y < \sqrt{\frac{3}{34}} \approx \sqrt{0.0882} \approx 0.297$. **Since**
 $\frac{1}{\sqrt{34}} \approx 0.17$, we have $0.17 \leq y < 0.297$. **In** this case,
 $f(34y^2) = 2 - 34y^2$. **We** want to solve $2 - 34y^2 = y$, **so**
 $34y^2 + y - 2 = 0$. Using the quadratic formula, we have
 $y = \frac{-1 \pm \sqrt{1 - 4(34)(-2)}}{2(34)} = \frac{-1 \pm \sqrt{273}}{68}$. **Since** $y > 0$, we take the
positive root, **so** $y = \frac{-1 + \sqrt{273}}{68}$.

(b) AIME 2025, Problem 21: amplification merges the setup into paragraphs.

Unedited (vanilla) — one fact per line

Let $2025 = 3^4 \cdot 5^2$.
The set A of positive integer divisors of 2025 is given by
 $A = \{3^a 5^b \mid 0 \leq a \leq 4, 0 \leq b \leq 2\}$.
The number of divisors of 2025 is $(4 + 1)(2 + 1) = 5 \cdot 3 = 15$.
Thus $|A| = 15$.
Let B be a randomly selected subset of A . The number of
possible subsets of A is 2^{15} .
We want to find the probability that B is a nonempty set
with the property that the least common multiple (LCM) of
its elements is 2025.

Amplified (sf = 3) — setup in connected paragraphs

Let A be the set of positive integer divisors of 2025. **We** first
find the prime factorization of 2025. **We** have $2025 = 5^2 \cdot 3^4$.
Therefore, the set A consists of all divisors of 2025, which
are of the form $5^a \cdot 3^b$, where $0 \leq a \leq 2$ and $0 \leq b \leq 4$. The
number of divisors of 2025 is $(2 + 1)(4 + 1) = 3 \cdot 5 = 15$. **So**,
 $|A| = 15$.
Let B be a randomly selected subset of A . **Since** B is
nonempty, the number of possible subsets of A is $2^{15} - 1$.
We want to find the probability that B is a nonempty set
such that the least common multiple (LCM) of its elements is
2025.

Table 7: Matched excerpts showing the newline effect in two AIME 2025 solutions; logical connectives are in bold.

Accuracy effects are modest in aggregate and heterogeneous across problems. Figure 13 reports both absolute final-answer accuracy and the per-problem change relative to Vanilla for AIME 2024 and 2025, with all three settings scored on identical matched samples. Aggregate effects are modest. The aggregate, however, understates the per-problem structure, because more than half of the problems are at a 0% floor under every setting and the remaining problems move in both directions. Amplification more than doubles accuracy on 2024 Problem 12 (17.3% $\rightarrow$ 41.3%) and adds 12.5 points on 2024 Problem 2 (70.3% $\rightarrow$ 82.8%), yet costs 4.7 points on 2024 Problem 23, where reversal is instead the beneficial direction (18.8% $\rightarrow$ 27.3%). No single setting is best on every problem: the intervention reliably changes how the model writes, and whether that helps depends on the problem.

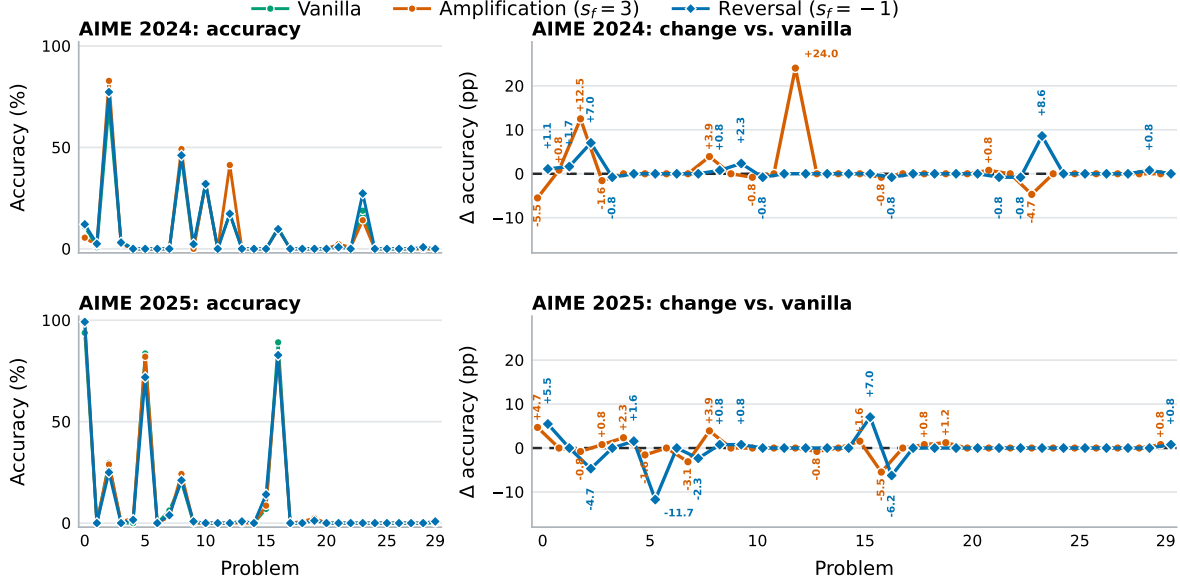


Figure 13: Per-problem AIME accuracy and intervention effects. The left panels show absolute accuracy for Vanilla, amplification, and reversal. The right panels show amplification and reversal minus Vanilla in percentage points, with the dashed black line marking zero change.

A representative trajectory-level effect. Figure 13 establishes that the accuracy effect is problem-dependent. Table 8 focuses on the clearest positive case, AIME 2024 Problem 12, where amplification raises accuracy from 17.3% to 41.3%. At the displayed period, the edit changes the top next-token prediction from a line break to **Since**; the sampled continuation then performs the hours-to-minutes conversion that a representative Vanilla failure omits. The clean/edit probability comparison is paired on an identical prefix within the amplified rollout. The Vanilla text comes from a separate rollout and is included as an illustrative failure, not as a common-prefix counterfactual completion.

Table 8: AIME 2024 Problem 12 case study. Amplification increases matched-sample accuracy from 17.3% to 41.3% (+24.0 percentage points) and supplies an omitted unit conversion in the representative trajectory.

Measured prefix: "... The time taken at $s + \frac{1}{2}$ km/h is $\frac{9}{3} = 3$ hours."	
Clean distribution; Vanilla failure At the measured prefix, top-1 is <code>\n</code> ($p=0.93$); Since has $p=0.03$. A separate Vanilla rollout (sample 104, incorrect) reads: "... The walk time is $\frac{9}{3} = 3$ hours. $\leftarrow$ The total time is $3 + 24 = 27$ minutes." Final Answer: <code>[27]</code> : hours were not converted.	Edited distribution and rollout (sf=3) At the same prefix, top-1 is Since ($p=0.41$), while <code>\n</code> falls to 0.37; the rollout samples Since : "... The time taken at $s + \frac{1}{2}$ km/h is $\frac{9}{3} = 3$ hours. Since the coffee shop takes $t = 24$ minutes, the total time taken is $3 \times 60 + 24 = 180 + 24 = 204$ minutes." Final Answer: <code>[204]</code> : the continuation performs the conversion.

7 Conclusion

The intervention isolates a direct behavioral role for L22:F31850: at sentence-ending periods, increasing its activation suppresses newline and stop behavior and promotes a restricted set of connective continuations, especially **Since** and **We**. Its effect on correctness, however, is conditional rather than uniformly beneficial. Continuing the argument helps when the unedited model breaks or concludes before a necessary step, as in AIME 2024 Problem 12, but can hurt when a problem benefits from committing to an answer or following a different trajectory. Consequently, a large and reproducible behavioral effect yields only a modest aggregate accuracy change and heterogeneous per-problem effects. The central distinction is therefore between controlling *how* the model continues its reasoning and improving *whether* it reaches the correct answer: L22:F31850 causally steers the former, but it is not a general-purpose accuracy knob.

A France–Paris Screening and Causal-Audit Protocol

A.1 Candidate extraction and controlled prompts

The screen starts from the saved attribution graph for **The capital of France is** with the correct next-token target **Paris**. Graph positions are indexed after excluding the beginning-of-sequence token, so the screened positions are **capital** at position 1, **France** at position 3, and the final **is** at position 4. We take the union of unique layer–feature coordinates whose positive aggregated Neumann attribution at any relevant position is greater than 0.01. This produces 1,034 candidates across all 34 layers.

The controlled suite contains 1,347 prompt–anchor records. The original graph prompt, **The capital of France is**, contributes three records because we read CLT activations separately at **capital**, **France**, and the final **is**. These three records are used only to check that the saved graph activations can be reproduced; they are not independent semantic-screening examples. For each of 16 country–capital pairs, the suite contains 12 neutral country prompts, 12 neutral capital prompts, 12 capital-to-country queries, 12 country-to-capital queries, 12 carried-country prompts, and 12 carried-capital prompts, for 1,152 records. Here *neutral* means that the entity appears in an ordinary sentence that neither asks about nor implies the capital relation; these prompts isolate activation at the literal entity token. For each of eight relations, the suite also contains 12 literal-relation prompts and 12 relation-query prompts, for another 192 records. The records sum to $3 + 1,152 + 192 = 1,347$. Table 9 gives one concrete example of every suite family.

Table 9: Representative controlled prompts used in the semantic screen. Each row shows one phrasing from a 12-template family, the controlled substitution, and the position at which CLT activation is read.

Suite family	Example	Controlled comparison	Readout
Neutral country	“Researchers discussed France yesterday.”	Replace France with each of the other 15 countries.	Country token
Neutral capital	“Researchers discussed Paris yesterday.”	Replace Paris with each of the other 15 capitals.	Capital token
Capital-to-country query	“The country whose capital is Paris is”	Replace Paris with each matched capital; the expected country changes with it.	Final position
Country-to-capital query	“The capital of France is”	Replace France with each matched country; the expected capital changes with it.	Final position
Carried country	“France was discussed earlier. The relevant country is”	Replace the earlier France mention with each matched country.	Final position
Carried capital	“Paris was discussed earlier. The relevant city is”	Replace the earlier Paris mention with each matched capital.	Final position
Literal relation	“Researchers discussed the capital of France yesterday.”	Replace capital with currency, language, continent, population, flag, climate, or leader.	Relation token
Relation query	“The capital of France is”	Make the same seven relation substitutions while holding France fixed.	Final position

The eight activation domains in Table 10 separate a response to the literal anchor token from

information inferred or carried to the final answer position. The statistical comparison units are the 16 countries, the corresponding 16 capitals, or the eight relations. The 1,347 prompt–anchor rows are controlled measurements over these units, not 1,347 independent semantic examples.

Table 10: The eight activation domains used in the semantic screen. Each domain compares the target condition with matched entities or relations at the same readout position.

Provisional role	Readout	Target–control comparison
Literal France	Named country token	France in neutral contexts versus the other 15 countries.
France inferred from Paris	Final position of country queries	Paris-to-France queries versus queries for the other country–capital pairs.
France carried forward	Final position after a country mention	Earlier mention of France versus earlier mentions of the other countries.
Literal Paris	Named city token	Paris in neutral contexts versus the other 15 capitals.
Paris answer state	Final position of capital queries	France-to-Paris queries versus queries for the other country–capital pairs.
Paris carried forward	Final position after a city mention	Earlier mention of Paris versus earlier mentions of the other capitals.
Literal capital	Named relation token	capital in neutral contexts versus the seven alternative relations.
Capital-query state	Final position of relation queries	Capital-of-France queries versus queries about the seven alternative relations.

A.2 Activation measurement and candidate rule

For every prompt, we run the original language model and capture the input to the MLP at the anchor position. We then apply the trained CLT encoder at the relevant layer using its affine encoder, JumpReLU threshold, and graph-matched per-position Top-400 sparsification. Thus the screen measures CLT dictionary coordinates on real model hidden states; it does not treat the coordinates as native MLP neurons.

Within each 12-template family, templates 1, 3, 5, 7, 9, 11 form the development split and templates 2, 4, 6, 8, 10, 12 form the held-out split. For each feature, domain, and split, we average activation over templates for every entity or relation. We then compare the target mean with the matched-control means. The reported standardized margin divides the target-minus-control mean by the standard deviation of the entity-level means within that domain.

A feature strictly passes a domain only if, on both splits, the target ranks first among the matched entities or relations, the target–control margin is positive, and the feature fires in at least 80% of target prompts. A provisional France or Paris candidate must pass at least two of its three domains and have positive graph attribution greater than 0.01 at the corresponding source position. A provisional **capital** candidate must pass both relation domains and exceed the same graph threshold at the **capital** position. These rules use both template splits to select candidates, so the held-out split checks cross-template consistency but is not an untouched final test set.

A.3 Matched causal-specificity protocol

Each provisional candidate is tested at its defining graph site by subtracting its measured activation times its baked cross-layer CLT decoder direction during a real Gemma-3-4B-IT forward. Recurrent

France and capital candidates are also tested at the final **is** position. The factorial suite crosses France, Germany, Italy, and Japan with capital, currency, language, and continent, and scores every cell on its own correct answer token. France candidates are compared with other-country facts, capital candidates with the three alternative relations, and Paris candidates with the other correct answers.

For each feature–site test, role support is the negative mean change in log probability of the correct answer over the proposed role contexts. The matched-control magnitude is the mean absolute change over the remaining contexts. We require role support greater than 0.05 and a support-minus-control margin greater than 0.05. We apply the same rule to joint ablations of all candidates assigned to each role.

None of the 15 individual tests or five joint role-group tests passes this criterion. The strongest late-state features affect capital answers, but their effects on currency, language, or continent answers are comparable or larger. This result does not show that the features lack causal effects; it shows that the intervention does not validate the narrower France-, Paris-, or capital-specific interpretation proposed by the activation screen.

A.4 Intervention bounds on the candidate shared mediator

We test whether **L33:F66785** carries the effects of the two retained input features. Ablating **L0:F26582** and **L0:F81670** reduces mediator activation by 6.57% and 8.03%, respectively; their joint ablation reduces it by 10.22%. Ablating the mediator changes the **Paris** logit by -0.0607 and $P(\text{Paris})$ by -0.0094 . These signs support **L33:F66785** as a causally relevant shared downstream feature.

Targeted rescue restores only the mediator activation lost under each source ablation. Table 11 bounds the resulting mediation claim: less than 2% of either source’s graph attribution passes through this mediator, and rescue recovers at most 2.3% of the measured source effect.

Table 11: Intervention bounds on the candidate shared **L33:F66785** mediator. Activation reduction is measured after source ablation; path share is the fraction of that source’s aggregate Paris attribution routed through the mediator; rescue restores only the source-induced loss in mediator activation.

Source ablation	Mediator activation reduction	Source path share	Rescued source effect
France feature	6.57%	1.95%	2.3%
Capital feature	8.03%	1.70%	0.6%
Joint sources	10.22%	–	0.1%

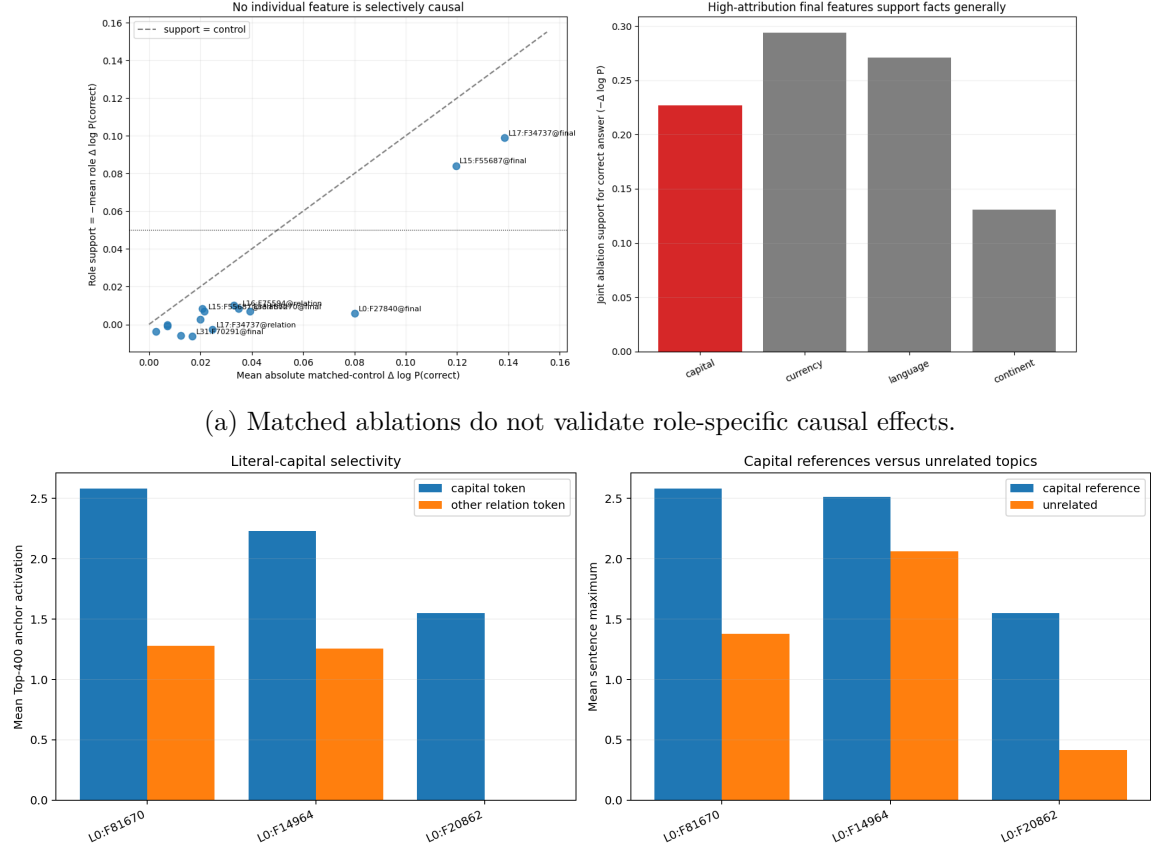
The candidate is therefore a genuine shared recipient but not a bottleneck. The small path shares and rescue fractions indicate that France–capital retrieval is distributed across several CLT coordinates.

A.5 Implementation provenance

The activation screen and causal follow-up use **unsloth/gemma-3-4b-it** with CLT checkpoint **Siyuc/converted_gemma3_100800**. Re-encoding the graph prompt closely reproduces the saved feature states: the median absolute activation error is 0.004883, the 99th percentile is 0.046875, and 98.3% of graph occurrences are within 0.03125. One TopK-boundary feature is a larger outlier.

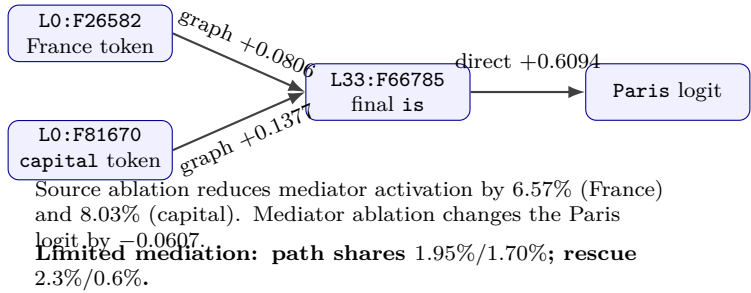
There is nevertheless a model-provenance gap in the output distribution. The available model assigns $P(\text{Paris}) = 0.810$ on the graph prompt, whereas the saved graph records 0.867. The original

weight source used to save the graph was not locally available during the follow-up. We therefore treat the activation screen as a close reproduction of the graph feature states, while interpreting all subsequent ablation and rescue magnitudes as deterministic tests in the available, closely matching model rather than an exact replay of the stored output distribution.



(b) Mean activation comparisons for three literal-capital candidates. Left: the **capital** anchor versus alternative relation anchors. Right: capital-reference versus unrelated sentence maxima.

Partial shared mediator



(c) A concrete shared path carries only a small effect share.

Figure 14: Intervention evidence for the France–Paris worked example. (a) Matched ablations do not support the narrow role labels suggested by activation. (b) Mean activation comparisons audit three literal-capital candidates against matched relation and unrelated controls. (c) A shared Paris-supporting coordinate carries only a small fraction of the source effects. Together, the evidence supports a partial, distributed trace rather than a complete feature-level explanation.

B Direct-Effect Computation Details

We write s for the layer at which the *source* node reads and r for the layer at which the *target* node reads, with corresponding token positions t_s and t_r . As before t indexes a token position and a indexes features. An attribution-graph node i is one of three types: a *feature node*, one per active (s, t, a) triple; a *token node*, one per non-special input token position t ; or a *logit node*, one per output logit k in the subset whose cumulative softmax probability reaches a target threshold (default 0.95). Every edge below is built from two ingredients: the *output vector* $\mathbf{v}_i^{\text{out}} \in \mathbb{R}^d$ that its source node *writes into* the residual stream, and the *input vector* $\mathbf{v}_i^{\text{in}} \in \mathbb{R}^d$, the direction along which its target node *reads* a scalar off the stream. A feature node has both vectors; a token node has only $\mathbf{v}_i^{\text{out}}$ (a pure source) and a logit node only $\mathbf{v}_i^{\text{in}}$ (a pure target).

For a feature node, $\mathbf{v}_i^{\text{out}}$ is the *activation-weighted* decoder contribution, spread cross-layer into every downstream output layer $\ell \geq s$, one block per write layer,

$$\mathbf{v}_{(s,t_s,a)}^{\text{out},\ell} = z_{s,t_s,a} \mathbf{w}_a^{s \rightarrow \ell}, \quad \ell \geq s, \quad (13)$$

where $z_{s,t_s,a}$ is the feature’s activation from equation (2) and $\mathbf{w}_a^{s \rightarrow \ell}$ is its $s \rightarrow \ell$ decoder row. For a token node it is the base-model embedding of the input token at position t ,

$$\mathbf{v}_t^{\text{out}} = \mathbf{W}_{\text{tok}_t}^E, \quad (14)$$

where $\mathbf{W}^E$ is the model’s token-embedding matrix and tok_t is the vocabulary index of the input token at position t .

For a feature node, $\mathbf{v}_i^{\text{in}}$ is the encoder row a of its layer r ,

$$\mathbf{v}_{(r,t_r,a)}^{\text{in}} = \mathbf{W}_r^{\text{enc}}[a, :], \quad (15)$$

where $\mathbf{W}_r^{\text{enc}}$ is the layer- r encoder matrix of equation (2). For a logit node it is the demeaned unembedding column,

$$\mathbf{v}_k^{\text{in}} = \mathbf{u}_k - \bar{\mathbf{u}}, \quad (16)$$

where $\mathbf{u}_k$ is the unembedding column for token k and $\bar{\mathbf{u}}$ its mean over the vocabulary. Demeaning removes the component shared across all logits, so the score reflects deviation from average output behavior rather than absolute logit magnitude.

We write $g_{i \rightarrow j}$ for the direct effect of source node i on target node j , the weight of the corresponding directed edge. Under the linearization,

$$g_{i \rightarrow j} = \mathbf{v}_i^{\text{out}} \cdot \frac{\partial (\mathbf{v}_j^{\text{in}} \cdot \mathbf{x}_j)}{\partial \mathbf{x}_i}, \quad (17)$$

where $\mathbf{x}_i$ is the residual stream at node i ’s location (the source position t_s), $\mathbf{x}_j$ is the residual stream at node j ’s location (the target position t_r), the partial derivative is the Jacobian of node j ’s scalar quantity $\mathbf{v}_j^{\text{in}} \cdot \mathbf{x}_j$ with respect to $\mathbf{x}_i$, and “ $\cdot$ ” sums over the d model dimensions. Because a feature’s output is spread over write layers (equation (13)), a feature-source edge sums the per-layer blocks, each derivative taken at the layer- ℓ MLP output at the source position, $\mathbf{m}_{\ell,t_s}$. For the feature-to-feature edge, with the target specialized to a feature node (r, t_r, a') and its encoder-row input vector of equation (15), this reads, fully indexed on both ends,

$$g_{(s,t_s,a) \rightarrow (r,t_r,a')} = \sum_{s \leq \ell < r} (z_{s,t_s,a} \mathbf{w}_a^{s \rightarrow \ell}) \cdot \frac{\partial (\mathbf{W}_r^{\text{enc}}[a', :] \cdot \mathbf{x}_{r,t_r})}{\partial \mathbf{m}_{\ell,t_s}}, \quad (18)$$

where the inner product $\mathbf{W}_r^{\text{enc}}[a', :] \cdot \mathbf{x}_{r,t_r}$ is the pre-activation of target feature a' read from the layer- r residual at the target position t_r , as in equation (2); it omits the constant biases $\mathbf{b}_r^{\text{pre}}$, $\mathbf{b}_r^{\text{enc}}$, which drop out of the derivative anyway. The sum stops at $\ell < r$ because a write at or after the target’s read layer never reaches the residual it reads. Substituting the logit input vector $\mathbf{v}_k^{\text{in}} = \mathbf{u}_k - \bar{\mathbf{u}}$ of equation (16) instead gives the feature-to-logit edge.

Rather than evaluating equation (18) independently for each edge, all incoming edges of a target are computed in a single backward pass: fixing the target j fixes the scalar $\mathbf{v}_j^{\text{in}} \cdot \mathbf{x}_j$, whose gradient with respect to the residual stream is available at every layer and position after one backward pass, and the inner product of this gradient with $\mathbf{v}_i^{\text{out}}$ at each source location yields the edges $g_{i \rightarrow j}$ from all sources.

In implementation (Figure 5), the forward pass caches the residual stream of every layer as a live computation-graph node, and the scalar readout is never materialized: the target’s input vector $\mathbf{v}_j^{\text{in}}$ is written directly into the residual-stream gradient at the target position t_r , with all other gradient seeds zero, so the backward pass propagates only the injected direction, and the per-source dot products fill the target’s row of attribution scores. These derivatives are taken on the original Gemma-3-4B-IT transformer, not on a CLT surrogate: each layer’s true MLP output $\mathbf{m}_{\ell,t}$ is a gradient leaf, and the CLT enters only through the direction vectors $\mathbf{v}^{\text{in}}$ and $\mathbf{v}^{\text{out}}$ of each feature.

References

- [1] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- [2] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. <https://matharena.ai>, 2025.
- [3] Adithya Bhaskar, Alexander Wettig, Dan Friedman, and Danqi Chen. Finding transformer circuits with edge pruning, 2024.
- [4] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models. <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>, 2023. OpenAI.
- [5] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features>.
- [6] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability, 2023.

- [7] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. *arXiv preprint arXiv:2406.11944*, 2024.
- [8] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- [9] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/toy_model/index.html.
- [10] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling, 2020. Copyright-filtered variant: monology/pile-uncopyrighted.
- [11] Gemma Team, Google DeepMind. Gemma 3 technical report, 2025. Base model: [google/gemma-3-4b-it](https://github.com/google/gemma-3-4b-it).
- [12] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, 2021.
- [13] Max Li, Samuel Marks, and Aaron Mueller. dictionary_learning. GitHub repository, 2023. https://github.com/saprmarks/dictionary_learning.
- [14] Jack Lindsey, Adly Templeton, Jonathan Marcus, Thomas Conerly, Joshua Batson, and Christopher Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/crosscoders/index.html>.
- [15] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [16] Mathematical Association of America. American invitational mathematics examination (AIME). <https://maa.org/maa-invitational-competitions/>, 2025.
- [17] Maxwell-Jia. AIME_2024 dataset. https://huggingface.co/datasets/Maxwell-Jia/AIME_2024, 2024.
- [18] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

- [19] Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. OpenWebMath: An open dataset of high-quality mathematical web text, 2023.
- [20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019. OpenAI Technical Report.
- [21] Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery, 2023.
- [22] Adly Templeton, Joshua Batson, Adam Jermyn, and Chris Olah. Predicting future activations. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/jan-update/index.html#predict-future>.
- [23] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>.
- [24] Transcoder Explore Project. Current clt training pipeline investigation, 2026. Local project file: `paper_investigation/stages/04_current_clt_training_pipeline.md`.
- [25] Transcoder Explore Project. Efficient transcoder network structure notes, 2026. Local project file: `paper_investigation/Efficient Transcoder.pdf`.
- [26] Transcoder Explore Project. Transcoder training pipeline overview, 2026. Local project file: `HydraTranscoder/docs/training_pipeline_overview.md`.
- [27] Transcoder Explore Project. V7 algorithm: Vanilla mainline, paired snapshot, and kv cache rewrite, 2026. Local project file: `notes/v7_algorithm.md`.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [29] Lei Yu, Jingcheng Niu, Zining Zhu, and Gerald Penn. Functional faithfulness in the wild: Circuit discovery with differentiable computation graph pruning, 2024.